LIVRO DE LÓGICA DE PROGRAMAÇÃO

Lógica de Programação com{ADVPL}

Do zero à aplicação no Protheus

Autor: Domingos Elias de Oliveira Neto

1ª edição
Outubro de 2026

Título: Lógica de Programação com ADVPL: do zero à aplicação no Protheus

Autor: Domingos Elias de Oliveira Neto

Edição: 1ª edição

Lançamento: outubro de 2026

© 2026 Domingos Elias de Oliveira Neto. Todos os direitos reservados. Nenhuma parte desta obra pode ser reproduzida sem autorização prévia do autor, exceto citações breves com indicação da fonte.

TOTVS, Protheus e ADVPL são marcas de propriedade da TOTVS S.A. e são citadas neste livro apenas para fins de identificação. Esta é uma obra independente, sem vínculo, patrocínio ou endosso da TOTVS.

Os exemplos de código têm finalidade didática. Teste sempre em ambiente de desenvolvimento ou homologação antes de usar em produção; o comportamento de algumas funções pode variar conforme a versão do ambiente.

Apresentação

Escrevi este livro para contribuir com a comunidade Protheus e ADVPL.

Sei como é difícil entender a lógica por trás do código quando se está começando do zero. Por isso, quis compartilhar um caminho mais claro, com conceitos explicados em português simples, exemplos comentados e exercícios, para que mais pessoas consigam aplicar tudo em programas reais.

Se este livro ajudar você, ou se tiver correções e sugestões, fale comigo. A comunidade cresce quando a gente aprende junto.

Domingos Elias de Oliveira Neto
Outubro de 2026

Sumário

Introdução **1**
Para quem é este livro 1
Como o livro está organizado 1
Convenções usadas 1
Como estudar com este livro 2
Sobre versões e documentação oficial 2

Capítulo 1 — O que é programar e o primeiro programa **3**
1.1 Programar é dar instruções precisas 3
1.2 O que é o ADVPL e onde ele roda 3
1.3 Seu primeiro programa 4
1.4 Mensagem sem tela: ConOut 4
1.5 Comentários 5

Capítulo 2 — Variáveis e tipos de dados **6**
2.1 O conceito 6
2.2 Declarando e atribuindo 6
2.3 Tipos de dados 6
2.4 Notação húngara: o prefixo que indica o tipo 7
2.5 Escopo: quem enxerga a variável 7
2.6 Convertendo entre tipos 8

Capítulo 3 — Operadores e expressões **9**
3.1 O conceito 9
3.2 Operadores aritméticos 9
3.3 Operadores de comparação 9
3.4 Operadores lógicos 10
3.5 Texto também tem operadores 11
3.6 Precedência: quem é calculado primeiro 11

Capítulo 4 — Decisões: If, Do Case e IIf **12**
4.1 O conceito 12
4.2 If / ElseIf / Else 12
4.3 Do Case 13
4.4 IIf: decisão em uma linha 13
4.5 Verificando se algo está vazio 13
4.6 Erros comuns 13

Capítulo 5 — Repetições: For e While **15**
5.1 O conceito 15
5.2 For / Next 15
5.3 While / EndDo 16

5.4 Exit e Loop 16
5.5 Laços dentro de laços 17

Capítulo 6 — Arrays 18

6.1 O conceito 18
6.2 Criando e acessando 18
6.3 Adicionando, inserindo e removendo 19
6.4 Percorrendo um array 19
6.5 Buscando e ordenando 19
6.6 Arrays dentro de arrays (matrizes) 20
6.7 Arrays são passados por referência 20

Capítulo 7 — Funções: dividindo o problema 22

7.1 O conceito 22
7.2 Tipos de função 22
7.3 Parâmetros e retorno 22
7.4 Validando parâmetros 23
7.5 Passagem por valor e por referência 23
7.6 Retornando mais de um valor 24
7.7 Como pensar em funções 24
7.8 Pontos de entrada 25

Capítulo 8 — Textos, datas e números 26

8.1 Por que isso importa 26
8.2 Funções de texto 26
8.3 Formatação 27
8.4 Datas 27
8.5 Números 28

Capítulo 9 — Tratamento de erros 29

9.1 O conceito 29
9.2 Prevenir 29
9.3 Capturar com Begin Sequence 30
9.4 O que fazer ao capturar um erro 30
9.5 Padrão útil: função que informa sucesso ou falha 31

Capítulo 10 — Acessando dados no Protheus 32

10.1 O conceito 32
10.2 Nomes e campos 32
10.3 Percorrendo uma tabela 33
10.4 Buscando um registro específico: DbSeek 34
10.5 Gravando: RecLock e MsUnlock 34
10.6 Consultas com SQL embutido 35
10.7 Qual usar: percorrer ou consultar? 36

Capítulo 11 — Projeto prático: juntando tudo 37

11.1 O problema 37
11.2 Pensando no algoritmo 37
11.3 O código 38
11.4 Refatorando em funções 39

11.5 Evoluindo o projeto 39

Capítulo 12 — Boas práticas e depuração 40

12.1 Escreva para o próximo leitor 40
12.2 Armadilhas clássicas (lista de conferência) 40
12.3 Depurando: como achar um erro 41
12.4 Testando sem medo 41
12.5 Desempenho em uma frase 41

Palavras finais 42

Apêndice A — Roteiro de estudos 43

Apêndice B — Glossário 44

Apêndice C — Gabarito comentado 45

Referências e leituras recomendadas 48

Sobre o autor 49

Introdução

Para quem é este livro

Este livro foi pensado para quem quer entender a lógica por trás do código, e não apenas copiar exemplos. Ele parte do zero: não exige que você já saiba programar. Se você trabalha, estuda ou pretende trabalhar com o ERP TOTVS Protheus, aqui encontrará a base necessária para escrever programas em ADVPL com segurança e para raciocinar sobre problemas antes de abrir o editor.

Como o livro está organizado

Cada capítulo segue a mesma estrutura: primeiro o **conceito** (a ideia, sem linguagem de programação), depois a **sintaxe em ADVPL**, depois um **exemplo comentado** e, por fim, **exercícios**.

Os capítulos 1 a 5 constroem a base da lógica: sequência, decisão e repetição. Os capítulos 6 a 9 ampliam as ferramentas: arrays, funções, textos, datas e tratamento de erros. Os capítulos 10 a 12 levam tudo para o dia a dia do Protheus: acesso a dados, um projeto completo e boas práticas. Ao final há um roteiro de estudos, um glossário e o gabarito comentado de exercícios selecionados.

Convenções usadas

Trechos de código aparecem em fonte monoespaçada, em blocos como o exemplo abaixo. Nomes de funções, comandos e campos, quando citados no texto, também usam essa fonte.

```
User Function Exemplo()
    ConOut("Isto é um bloco de código")
Return Nil
```

Ao longo do texto, caixas destacadas chamam a atenção para pontos específicos:

> **Dica:** sugestão que economiza tempo ou melhora o seu código.

> **Atenção:** armadilha comum ou ponto que costuma gerar erro.

Os exemplos usam a convenção de nomes mais adotada na comunidade Protheus (notação húngara), explicada no Capítulo 2.

Como estudar com este livro

Uma regra de ouro: **não leia apenas, digite**. Abra o seu ambiente de desenvolvimento (TDS, VS Code com a extensão TOTVS ou o que você usa), digite cada exemplo, execute, quebre de propósito e conserte. Lógica de programação se aprende com as mãos. Tente resolver cada exercício antes de consultar o gabarito.

Sobre versões e documentação oficial

Funções e comportamentos podem variar conforme a versão do Application Server e da biblioteca (lib) do seu ambiente. Quando houver risco disso, o texto avisa. A documentação oficial da TOTVS, o TDN (TOTVS Developer Network), é a referência definitiva: na dúvida, confirme lá antes de colocar um código em produção.

CAPÍTULO 1

O que é programar e o primeiro programa

> **Neste capítulo:** você entende o que é um algoritmo, conhece os três blocos da lógica (sequência, decisão e repetição) e escreve o seu primeiro programa em ADVPL.

1.1 Programar é dar instruções precisas

Um computador é extremamente rápido e extremamente literal: ele faz exatamente o que você mandou, na ordem em que mandou, nem mais nem menos. Programar é escrever essa lista de instruções. Uma lista de passos que resolve um problema chama-se **algoritmo**.

Exemplo de algoritmo do dia a dia, "preparar café":

1. Ferver a água.
2. Colocar o pó no filtro.
3. Se a água ferveu, despejar sobre o pó.
4. Servir.

Repare que já aparecem os três blocos que formam **toda** a lógica de programação:

- **Sequência**: os passos acontecem em ordem (1, 2, 3, 4).
- **Decisão**: "se a água ferveu..." (um caminho ou outro).
- **Repetição**: "enquanto a água não ferver, aguardar" (fazer de novo até uma condição mudar).

Tudo o que você vai aprender neste livro é uma forma de expressar sequência, decisão e repetição, mais maneiras de guardar dados (variáveis e arrays) e de organizar o código (funções).

1.2 O que é o ADVPL e onde ele roda

O **ADVPL** (Advanced Protheus Language) é a linguagem usada para customizar e estender o ERP Protheus, da TOTVS. Seu código é compilado e guardado no **RPO** (repositório de objetos) e executado pelo **Application Server**. Isso significa que seu programa roda dentro do Protheus e tem acesso fácil às tabelas, aos parâmetros e às telas do sistema.

A linguagem é derivada da família xBase (Clipper), por isso você verá comandos como DbSeek, DbSkip e Eof. Existe também o **TLPP**, evolução da linguagem com recursos modernos; os conceitos de lógica deste livro valem para os dois.

1.3 Seu primeiro programa

```
#Include "TOTVS.CH"

User Function OlaMundo()
    MsgInfo("Olá, Protheus!", "Meu primeiro programa")
Return Nil
```

Vamos ler linha a linha:

- **#Include "TOTVS.CH"** traz definições e comandos prontos (é como "importar" um conjunto de ferramentas). Quase todo fonte começa assim.
- **User Function OlaMundo()** declara uma função do usuário. O prefixo User faz com que ela seja chamada como U_OlaMundo e evita conflito com funções padrão da TOTVS.
- **MsgInfo(...)** exibe uma janela de mensagem. Só funciona quando há interface (SmartClient).
- **Return Nil** encerra a função. Toda função termina com Return.

> **Atenção:** tradicionalmente só os primeiros 10 caracteres do nome de uma função são significativos, então dois nomes longos que começam igual podem colidir. Prefira nomes curtos e distintos e confira o limite na documentação da sua versão.

1.4 Mensagem sem tela: ConOut

Nem todo programa tem tela. Rotinas agendadas (jobs) e processos de bastidor não têm interface, e MsgInfo não funciona neles. Para registrar o que está acontecendo, use o console do servidor:

```
User Function TesteLog()
    ConOut("Iniciando processamento...")
    ConOut("Fim.")
Return Nil
```

O ConOut será seu melhor amigo para entender o que o programa está fazendo, principalmente no começo.

1.5 Comentários

Comentários são notas para humanos; o compilador os ignora.

```
// comentário de uma linha

/* comentário
   de várias linhas */
```

Comente o **porquê** das decisões, não o óbvio. nTotal += 1 // soma 1 não ajuda ninguém; // desconta o frete antes de aplicar o desconto ajuda muito.

Exercícios do Capítulo 1

1. Escreva em português, em passos numerados, o algoritmo para "trocar uma lâmpada". Marque onde há sequência, decisão e repetição.
2. Crie uma User Function que mostre com ConOut o seu nome e a sua cidade.
3. Altere o OlaMundo para mostrar duas mensagens seguidas. O que acontece com a ordem?

CAPÍTULO 2

Variáveis e tipos de dados

> **Neste capítulo:** você aprende a guardar valores em variáveis, conhece os tipos de dados do ADVPL, entende o escopo e converte valores entre tipos.

2.1 O conceito

Uma **variável** é uma "caixa" com nome, guardada na memória, onde você coloca um valor para usar depois. O nome permite achar a caixa; o valor pode mudar ao longo do programa (daí "variável").

Sem variáveis, um programa só trabalharia com valores fixos escritos no código. Com elas, o mesmo programa processa qualquer cliente, qualquer pedido, qualquer data.

2.2 Declarando e atribuindo

```
User Function Variaveis()
    Local cNome  := "Maria"
    Local nIdade := 32
    Local lAtivo := .T.

    ConOut(cNome)
Return Nil
```

- Local **declara** a variável (cria a caixa).
- := **atribui** um valor (coloca algo na caixa). Leia como "recebe".
- As declarações ficam **no início** da função, antes dos comandos executáveis.
- Uma variável declarada sem valor começa como Nil (ausência de valor).

2.3 Tipos de dados

O ADVPL é de tipagem dinâmica: a variável assume o tipo do valor que recebe e pode mudar de tipo, o que dá flexibilidade e também permite erros. Os tipos que você mais usará:

Tipo	Letra	Exemplo	Uso
Caractere	C	"Maria"	textos, códigos
Numérico	N	10, 3.14	quantidades, valores
Data	D	Date()	datas
Lógico	L	.T. / .F.	verdadeiro ou falso
Array	A	{1, 2, 3}	listas
Bloco de código	B	{|x| x * 2}	trechos de código guardados
Objeto	O	TFont():New(...)	objetos
Nulo	U	Nil	sem valor

Para descobrir o tipo de uma variável, use ValType():

```
Local nValor := 10
ConOut(ValType(nValor))    // N
```

> **Atenção:** códigos numéricos costumam ser guardados como texto no Protheus (por exemplo, "000123"), porque os zeros à esquerda importam. Nunca assuma que um campo "que parece número" é numérico; olhe o dicionário de dados.

2.4 Notação húngara: o prefixo que indica o tipo

Por convenção, o nome começa com uma letra que indica o tipo: cNome (caractere), nTotal (numérico), dEmissao (data), lOk (lógico), aLista (array), bBloco (bloco), oObj (objeto), xGeral (qualquer tipo). Isso parece detalhe, mas evita muitos erros: ao ler nValor + cNome, você já enxerga o problema.

2.5 Escopo: quem enxerga a variável

O **escopo** define onde a variável existe:

- **Local**: existe só dentro da função onde foi declarada. É a escolha padrão, a mais segura e a mais rápida.
- **Static**: existe no arquivo-fonte inteiro e mantém o valor entre chamadas.
- **Private**: visível na função que a criou e em todas as funções chamadas por ela.
- **Public**: visível em toda a thread. Evite: qualquer parte do sistema pode alterá-la e o rastreio de erros fica difícil.

> **Regra prática:** use Local por padrão. Só use outro escopo quando souber exatamente por quê. Você ainda verá Private e Public em código antigo e em pontos de entrada, e é importante saber reconhecê-los.

2.6 Convertendo entre tipos

Você não pode juntar texto com número diretamente; precisa converter:

```
Local nQtd := 5
Local cMsg := "Quantidade: " + cValToChar(nQtd)   // "Quantidade: 5"
Local nNum := Val("42")                            // 42
Local cDat := DToC(Date())                         // ex.: "17/10/2026"
Local dDat := SToD("20261017")                     // data
```

- **cValToChar(x):** converte qualquer valor em texto (a mais prática).
- **Val(c):** texto para número.
- **DToC(d):** data para texto formatado. DToS(d): data para AAAAMMDD, formato guardado nas tabelas.
- **SToD(c):** texto AAAAMMDD para data.

No banco do Protheus, datas ficam como texto AAAAMMDD; por isso SToD e DToS aparecem o tempo todo.

Exercícios do Capítulo 2

1. Declare variáveis para nome, idade, salário e se o funcionário está ativo. Dê a cada uma o prefixo correto.
2. O que acontece ao executar "Total: " + 10? Teste e leia a mensagem de erro. Depois corrija.
3. Explique com suas palavras por que Local é preferível a Public.

CAPÍTULO 3

Operadores e expressões

> **Neste capítulo:** você usa operadores aritméticos, de comparação e lógicos, evita a armadilha de = versus == e aprende a ordem em que as contas são feitas.

3.1 O conceito

Uma **expressão** é qualquer coisa que produz um valor: 2 + 3, cNome, nA > nB. Os **operadores** combinam valores em expressões. Dominá-los é o que permite fazer o programa "pensar".

3.2 Operadores aritméticos

Operador	Significado	Exemplo	Resultado
+	soma	7 + 2	9
-	subtração	7 - 2	5
*	multiplicação	7 * 2	14
/	divisão	7 / 2	3.5
%	resto da divisão	7 % 2	1
** ou ^	potência	2 ** 3	8

Atalhos de atribuição, muito comuns:

```
nTotal += 10   // nTotal := nTotal + 10
nTotal -= 2    // nTotal := nTotal - 2
nContador++    // soma 1
```

3.3 Operadores de comparação

Comparações produzem um valor **lógico** (.T. ou .F.):

Operador	Significado
==	igual (exato)
<>, != ou #	diferente
>, <	maior, menor
>=, <=	maior ou igual, menor ou igual
$	está contido ("A" $ "ABC" é .T.)

> **Atenção:** a armadilha mais famosa do ADVPL é = versus ==. Em uma comparação de textos, o = simples, com a configuração padrão do ambiente (SET EXACT OFF), compara só até o tamanho do lado direito. Assim, "ABC" = "AB" pode dar verdadeiro! Já o == exige igualdade completa. **Para comparar, use sempre ==.** E lembre: := atribui, == compara. Confundir os dois é o erro de iniciante número um.

3.4 Operadores lógicos

Combinam condições:

- **.And. (e):** verdadeiro só se **as duas** forem verdadeiras.
- **.Or. (ou):** verdadeiro se **pelo menos uma** for verdadeira.
- **.Not. ou ! (não):** inverte o valor.

```
If nIdade >= 18 .And. lAtivo
    ConOut("Pode prosseguir")
EndIf
```

Tabela-verdade (memorize a ideia, não a tabela):

A	B	A .And. B	A .Or. B
.T.	.T.	.T.	.T.
.T.	.F.	.F.	.T.
.F.	.T.	.F.	.T.
.F.	.F.	.F.	.F.

3.5 Texto também tem operadores

O + concatena (junta) textos. O - também junta, mas move os espaços do final do primeiro texto para o fim do resultado:

```
Local cA := "Ana  "
Local cB := "Silva"
ConOut(cA + cB)   // "Ana  Silva"
ConOut(cA - cB)   // "AnaSilva  "
```

3.6 Precedência: quem é calculado primeiro

Como na matemática, multiplicação vem antes da soma. Na dúvida, **use parênteses**. Eles deixam a intenção clara para quem lê:

```
nRes := 2 + 3 * 4       // 14
nRes := (2 + 3) * 4     // 20
```

Exercícios do Capítulo 3

1. Calcule, com variáveis, o valor final de um produto com 10% de desconto e 5% de imposto sobre o valor com desconto.
2. Sem executar, diga o resultado de .T. .And. .F. .Or. .T.. Depois execute e confira. (Dica: use parênteses para deixar a ordem explícita.)
3. Escreva uma expressão que seja verdadeira quando nNota está entre 0 e 10, inclusive.

CAPÍTULO 4

Decisões: If, Do Case e IIf

> **Neste capítulo:** você faz o programa escolher caminhos com If, Do Case e IIf, e aprende a ordenar as condições corretamente.

4.1 O conceito

Programa que sempre faz a mesma coisa é pouco útil. A **decisão** permite que o programa escolha um caminho conforme uma condição. Pense em "se estiver chovendo, levo guarda-chuva; senão, não levo".

4.2 If / ElseIf / Else

```
User Function Situacao()
    Local nNota := 6.5
    Local cSit  := ""

    If nNota >= 7
        cSit := "Aprovado"
    ElseIf nNota >= 5
        cSit := "Recuperação"
    Else
        cSit := "Reprovado"
    EndIf

    ConOut("Situação: " + cSit)
Return Nil
```

Como ler: o programa testa as condições **de cima para baixo** e executa o **primeiro** bloco verdadeiro; os demais são ignorados. Por isso a ordem importa: se você testasse nNota >= 5 antes de nNota >= 7, ninguém seria aprovado!

Todo If precisa fechar com EndIf. ElseIf e Else são opcionais.

4.3 Do Case

Quando há muitas alternativas para a mesma variável, Do Case fica mais legível que vários ElseIf:

```
Do Case
    Case cUF == "MG"
        nFrete := 20
    Case cUF == "SP" .Or. cUF == "RJ"
        nFrete := 30
    Otherwise
        nFrete := 50
EndCase
```

Otherwise é o “nenhum dos casos anteriores”, equivalente ao Else.

4.4 IIf: decisão em uma linha

IIf(condição, valor_se_verdadeiro, valor_se_falso) devolve um valor. É ótimo para escolhas simples, ruim para lógica longa:

```
cTipo := IIf(nIdade >= 18, "Adulto", "Menor")
```

4.5 Verificando se algo está vazio

Uma decisão muito comum é saber se um valor existe. A função Empty() responde .T. para texto vazio ou só com espaços, número zero, data vazia, array sem elementos e Nil:

```
If Empty(cCodigo)
    ConOut("Informe o código")
EndIf
```

4.6 Erros comuns

- Usar = esperando comparar com exatidão (use ==).
- Esquecer o EndIf.
- Colocar condições na ordem errada (a primeira verdadeira “vence”).
- Testar duas vezes o que poderia ser um Else.

Exercícios do Capítulo 4

1. Dado o valor de uma compra, aplique 5% de desconto se for maior ou igual a 100, 10% se for maior ou igual a 500 e 15% se for maior ou igual a 1000. Qual a ordem correta dos testes?
2. Reescreva o exercício anterior usando Do Case.
3. Dado um ano, descubra se é bissexto (divisível por 4, exceto os divisíveis por 100, a menos que também sejam divisíveis por 400). Dica: use %.

CAPÍTULO 5

Repetições: For e While

> **Neste capítulo:** você repete blocos de código com For e While, usa acumuladores, controla laços com Exit e Loop e evita laços infinitos.

5.1 O conceito

Se você precisasse imprimir os números de 1 a 1000, não escreveria mil linhas. A **repetição** (ou laço) executa um bloco várias vezes. Existem dois grandes tipos:

- **Quando você sabe quantas vezes repetir**: use For.
- **Quando repete até uma condição mudar**: use While.

5.2 For / Next

```
User Function Contagem()
    Local nI := 0

    For nI := 1 To 5
        ConOut("Volta número " + cValToChar(nI))
    Next nI
Return Nil
```

O For cria um contador (nI), parte do valor inicial, executa o bloco e soma 1 até passar do limite. Para pular de 2 em 2, ou contar de trás para frente, use Step:

```
For nI := 10 To 0 Step -2
    ConOut(nI)    // 10, 8, 6, 4, 2, 0
Next nI
```

Exemplo clássico: somar de 1 a 100.

```
Local nI    := 0
Local nSoma := 0

For nI := 1 To 100
    nSoma += nI
Next nI

ConOut("Soma: " + cValToChar(nSoma))   // 5050
```

O padrão aqui é o **acumulador**: uma variável que começa em zero e vai juntando valores a cada volta. Você o usará em totais, contagens e somatórios por toda a vida.

5.3 While / EndDo

```
Local nSaldo := 100

While nSaldo > 0
    nSaldo -= 30
    ConOut("Saldo: " + cValToChar(nSaldo))
EndDo
```

O While testa a condição **antes** de cada volta. Se ela já começar falsa, o bloco nem roda.

> **Atenção:** cuidado com o laço infinito. Se nada dentro do While muda a condição, o programa nunca termina e pode travar a thread do servidor. Antes de rodar, pergunte: "o que faz esta condição ficar falsa?"

O While é o laço natural para percorrer registros de uma tabela: "enquanto não for o fim do arquivo, processe o registro e vá para o próximo" (veremos no Capítulo 10).

5.4 Exit e Loop

- **Exit: sai** do laço imediatamente.
- **Loop: pula** o resto da volta atual e vai para a próxima.

```
For nI := 1 To 10
    If nI % 2 == 0
        Loop        // pula os pares
    EndIf
    If nI > 7
        Exit        // para tudo
    EndIf
    ConOut(nI)      // 1, 3, 5, 7
Next nI
```

5.5 Laços dentro de laços

Um laço pode conter outro. Para uma tabuada completa:

```
For nI := 1 To 10
    For nJ := 1 To 10
        cLinha := cValToChar(nI) + " x " + cValToChar(nJ)
        ConOut(cLinha + " = " + cValToChar(nI * nJ))
    Next nJ
Next nI
```

O laço de dentro roda **completo** a cada volta do de fora (aqui, 10 x 10 = 100 execuções). Aninhar muito deixa o programa lento; mantenha isso em mente quando houver consultas ao banco dentro de laços.

Exercícios do Capítulo 5

1. Mostre os números pares de 1 a 50.
2. Calcule o fatorial de um número (5! = 5 × 4 × 3 × 2 × 1).
3. Dado um saldo inicial de 1000 e um rendimento de 1% ao mês, descubra em quantos meses o saldo passa de 1500. Qual laço é o mais adequado aqui, e por quê?
4. Escreva os números de 1 a 20 trocando por “Fizz” os múltiplos de 3, por “Buzz” os de 5 e por “FizzBuzz” os de ambos.

CAPÍTULO 6

Arrays

> **Neste capítulo:** você guarda listas de valores em arrays, percorre, busca e ordena elementos, trabalha com matrizes e entende por que arrays são passados por referência.

6.1 O conceito

Uma variável guarda **um** valor. Mas e se você precisa guardar a lista de produtos de um pedido, ou os 50 clientes de uma consulta? Criar 50 variáveis não faz sentido. O **array** é uma variável que guarda **vários valores em posições numeradas**, como uma prateleira com gavetas numeradas a partir de 1.

No Protheus, arrays estão em todo lugar: itens de pedidos, parâmetros de rotinas automáticas, retornos de funções, colunas de grids. Dominar arrays é um dos maiores saltos de qualidade que você dará.

6.2 Criando e acessando

```
Local aFrutas := {"Maçã", "Banana", "Uva"}

ConOut(aFrutas[1])        // Maçã (a primeira posição é 1, não 0)
ConOut(Len(aFrutas))      // 3

aFrutas[2] := "Laranja"   // altera a posição 2
```

Pontos importantes:

- Os índices começam em **1**.
- Len(aArray) devolve a quantidade de elementos.
- Acessar uma posição que não existe (por exemplo aFrutas[10] com 3 elementos) gera erro de execução (array out of bounds). Confira o Len antes quando houver dúvida.

- Um array pode misturar tipos: {"Maria", 32, .T.}.

6.3 Adicionando, inserindo e removendo

```
Local aLista := {}              // array vazio

aAdd(aLista, "A")               // {"A"}
aAdd(aLista, "B")               // {"A", "B"}
aAdd(aLista, "C")               // {"A", "B", "C"}

aDel(aLista, 2)                 // remove a posição 2, mas mantém o tamanho
aSize(aLista, Len(aLista) - 1)  // ajusta o tamanho depois do aDel
```

O aDel tem uma pegadinha: ele **desloca** os elementos seguintes e deixa o último como Nil, sem diminuir o array. Por isso se usa o par aDel + aSize para realmente remover.

- **aAdd(aArray, xValor):** acrescenta ao final.
- **aIns(aArray, nPos):** abre espaço (o conteúdo da posição fica Nil) e empurra os demais.
- **Array(n):** cria um array com n posições vazias.

6.4 Percorrendo um array

A combinação mais usada em todo o ADVPL é **For + Len**:

```
Local aNomes := {"Ana", "Bruno", "Carla"}
Local nI     := 0

For nI := 1 To Len(aNomes)
    ConOut(cValToChar(nI) + " - " + aNomes[nI])
Next nI
```

6.5 Buscando e ordenando

```
Local aNomes := {"Carla", "Ana", "Bruno"}
Local nPos   := 0

nPos := aScan(aNomes, "Ana")    // 2 (0 se não achar)
aSort(aNomes)                   // {"Ana", "Bruno", "Carla"}
```

aScan devolve a **posição** do elemento ou **0** se não encontrar. Esse zero é a base de uma decisão muito comum ("se não achou, inclui"), que você verá no projeto do Capítulo 11.

6.6 Arrays dentro de arrays (matrizes)

Cada elemento de um array pode ser outro array. É assim que se representam tabelas (linhas e colunas):

```
Local aItens := {;
    {"001", "Camiseta", 10},;
    {"002", "Calça",     5},;
    {"003", "Boné",     20};
}

ConOut(aItens[2][2])      // Calça (linha 2, coluna 2)

For nI := 1 To Len(aItens)
    ConOut(aItens[nI][1] + " - " + aItens[nI][2])
Next nI
```

Leia aItens[2][2] como "vá ao elemento 2 de aItens e, dentro dele, pegue o elemento 2". O ponto e vírgula (`;`) no fim das linhas é só a continuação da linha na declaração.

Para buscar dentro de uma matriz, use `aScan` com um **bloco de código**:

```
nPos := aScan(aItens, {|x| x[1] == "002"})   // busca o código na coluna 1
```

O bloco {|x| x[1] == "002"} é uma mini-função anônima: para cada elemento x, devolve .T. ou .F.. O aScan para no primeiro .T..

6.7 Arrays são passados por referência

Este é um conceito que pega muita gente. Quando você passa um array para uma função, a função recebe **o mesmo array**, não uma cópia. Se ela alterar, o original muda também. Para trabalhar com uma cópia independente, use aClone:

```
Local aOriginal := {1, 2, 3}
Local aCopia    := aClone(aOriginal)

aCopia[1] := 99
ConOut(aOriginal[1])    // 1  (não mudou)
```

Se em vez de aClone você escrevesse aCopia := aOriginal, as duas variáveis apontariam para o mesmo array e alterar uma alteraria a outra.

Exercícios do Capítulo 6

1. Crie um array com 5 números e calcule a soma e a média.
2. Encontre o maior e o menor valor de um array sem usar Max/Min sobre o array inteiro (use laço e comparação).
3. Dado um array de nomes, mostre-os em ordem alfabética.
4. Monte uma matriz com código, descrição e preço de 3 produtos e mostre o produto mais caro.
5. Teste a diferença entre aCopia := aOriginal e aCopia := aClone(aOriginal) alterando um elemento.

CAPÍTULO 7

Funções: dividindo o problema

> **Neste capítulo:** você divide programas grandes em funções, passa parâmetros, devolve resultados, valida entradas e entende a passagem por valor e por referência.

7.1 O conceito

Problemas grandes ficam fáceis quando divididos em problemas pequenos. Uma **função** é um bloco de código com nome que recebe dados (**parâmetros**), faz uma tarefa e devolve um resultado (**retorno**). Você escreve uma vez e usa quantas vezes precisar.

Benefícios: o código fica mais curto, mais legível, mais fácil de testar e de corrigir, porque um erro é corrigido em um lugar só.

7.2 Tipos de função

- **User Function**: pode ser chamada de outros programas e pelo Protheus (menus, pontos de entrada). O nome externo ganha o prefixo U_.
- **Static Function**: só pode ser chamada **dentro do mesmo arquivo-fonte**. É a escolha para funções auxiliares internas e evita conflito de nomes.
- **Function**: função padrão, reservada à TOTVS; evite criar com esse tipo nos seus fontes.

7.3 Parâmetros e retorno

```
#Include "TOTVS.CH"

User Function TesteSoma()
    Local nResultado := Soma(10, 5)
    ConOut("Resultado: " + cValToChar(nResultado))   // 15
Return Nil

Static Function Soma(nA, nB)
    Local nTotal := nA + nB
Return nTotal
```

O valor depois de Return é o que a função devolve a quem a chamou. Uma função sem nada a devolver costuma terminar com Return Nil.

7.4 Validando parâmetros

Em ADVPL, o tipo do parâmetro não é imposto: alguém pode chamar Soma("a", 5) e o programa quebra. Proteja suas funções:

```
Static Function Dobro(nValor)
    If ValType(nValor) <> "N"
        ConOut("Dobro: parâmetro inválido")
        Return 0
    EndIf
Return nValor * 2
```

7.5 Passagem por valor e por referência

Por padrão, números, textos, datas e lógicos são passados **por valor**: a função recebe uma cópia, e mudar dentro dela não afeta o original. Para que a função altere a variável de quem chamou, use `@` na chamada:

```
User Function TesteRef()
    Local nValor := 10

    Acrescenta(nValor)      // por valor: não muda
    ConOut(nValor)          // 10

    Acrescenta(@nValor)     // por referência: muda
    ConOut(nValor)          // 11
Return Nil

Static Function Acrescenta(nNum)
    nNum++
Return Nil
```

Lembre do Capítulo 6: arrays já se comportam como referência, mesmo sem o `@`.

7.6 Retornando mais de um valor

Uma função devolve um único valor, mas esse valor pode ser um **array**. Assim você devolve vários resultados de uma vez:

```
Static Function MinMax(aNumeros)
    Local nMin := aNumeros[1]
    Local nMax := aNumeros[1]
    Local nI   := 0

    For nI := 2 To Len(aNumeros)
        If aNumeros[nI] < nMin
            nMin := aNumeros[nI]
        EndIf
        If aNumeros[nI] > nMax
            nMax := aNumeros[nI]
        EndIf
    Next nI
Return {nMin, nMax}
```

7.7 Como pensar em funções

Uma boa função faz **uma coisa** e tem um nome que diz qual. Se você precisa de "e" para descrever o que ela faz ("valida o cliente **e** grava o pedido **e** envia e-mail"), provavelmente são três funções. Uma regra prática: se a função não cabe em uma tela, divida.

Um bom exercício mental é escrever primeiro o **roteiro principal**, chamando funções que ainda não existem, e depois implementá-las:

```
User Function ProcPedido()
    If !ValidaCliente()
        Return Nil
    EndIf
    CalculaTotais()
    GravaPedido()
Return Nil
```

Você lê o fluxo inteiro em segundos. Os detalhes ficam cada um no seu lugar.

7.8 Pontos de entrada

No Protheus, os **pontos de entrada** são funções com nomes pré-definidos que o sistema padrão chama em momentos específicos (por exemplo, antes de gravar um documento). Se a sua User Function tiver o nome esperado, o Protheus a executa e usa o retorno dela. Tudo o que você aprendeu aqui (parâmetros, retorno, validação) é a base para escrever pontos de entrada com segurança; a documentação de cada um, no TDN, diz quais parâmetros chegam e o que deve ser devolvido.

Exercícios do Capítulo 7

1. Crie Static Function Media(aNumeros) que devolva a média de um array.
2. Crie Static Function EhPar(nNum) que devolva .T. ou .F. e use-a em um laço de 1 a 20.
3. Escreva uma função Fatorial(nN) e outra Combinacoes(nN, nK) que usa a primeira.
4. Crie uma função que troque o valor de duas variáveis usando passagem por referência.

CAPÍTULO 8

Textos, datas e números

> **Neste capítulo:** você domina as funções do dia a dia para manipular textos, formatar valores, calcular com datas e arredondar números.

8.1 Por que isso importa

A maior parte dos programas no Protheus é **manipulação de dados**: montar códigos, formatar valores, comparar datas, limpar textos. As funções abaixo são o seu kit de ferramentas diário.

8.2 Funções de texto

Função	O que faz	Exemplo	Resultado
Len(c)	tamanho	Len("Ana")	3
Upper(c) / Lower(c)	maiúsculas / minúsculas	Upper("ana")	“ANA”
AllTrim(c)	tira espaços das pontas	AllTrim(" Ana ")	“Ana”
Left(c, n) / Right(c, n)	n primeiros / últimos	Left("Protheus", 4)	“Prot”
SubStr(c, ini, qtd)	pedaço do texto	SubStr("Protheus", 5, 3)	“the”
At(busca, c)	posição de um texto dentro de outro (0 se não achar)	At("t", "Protheus")	3
StrTran(c, de, para)	troca ocorrências	StrTran("a-b", "-", "/")	“a/b”
PadR(c, n) / PadL(c, n)	completa com espaços à direita / esquerda até n	PadR("Ana", 6)	“Ana “

Função	O que faz	Exemplo	Resultado
Replicate(c, n)	repete n vezes	Replicate("-", 5)	"-----"

> **Atenção:** campos de texto vêm do banco preenchidos com espaços até o tamanho do campo. Por isso comparar SA1->A1_NOME == "Maria" quase nunca dá certo; o correto é AllTrim(SA1->A1_NOME) == "Maria".

Zeros à esquerda em códigos (como "000045") são obtidos com StrZero:

```
cCod := StrZero(45, 6)   // "000045"
```

8.3 Formatação

Transform formata um valor segundo uma máscara (*picture*):

```
Local nValor := 1234567.89
ConOut(Transform(nValor, "@E 9,999,999.99"))   // 1.234.567,89
```

O @E pede o formato brasileiro (ponto para milhar, vírgula para decimais).

8.4 Datas

Datas são um tipo próprio, e dá para fazer contas com elas:

```
Local dHoje := Date()            // data do sistema
Local dVenc := dHoje + 30        // daqui a 30 dias
Local nDias := dVenc - dHoje     // 30 (diferença em dias)

ConOut(DToC(dVenc))                // dd/mm/aaaa (conforme o ambiente)
ConOut(DToS(dVenc))                // aaaammdd
ConOut(cValToChar(Day(dVenc)))     // dia
ConOut(cValToChar(Month(dVenc)))   // mês
ConOut(cValToChar(Year(dVenc)))    // ano
```

Somar ou subtrair um número a uma data move a data em dias. Subtrair duas datas devolve a quantidade de dias entre elas.

> **Dica:** Date() é a data do sistema operacional do servidor, enquanto dDataBase é a data base do Protheus (a que o usuário escolhe ao entrar). Para regras de negócio, costuma ser a dDataBase a data correta.

8.5 Números

```
Round(3.14159, 2)    // 3.14
Int(7.9)             // 7 (corta a parte decimal)
Abs(-5)              // 5
Mod(10, 3)           // 1 (resto)
Max(3, 8)            // 8
Min(3, 8)            // 3
```

> **Atenção:** em valores monetários, arredonde explicitamente com Round no ponto certo do cálculo. Pequenas diferenças somadas em milhares de linhas viram diferença de centavos no fechamento.

Exercícios do Capítulo 8

1. Dado um CPF como texto com pontos e traço, remova a pontuação usando StrTran.
2. Dado um nome completo, mostre só o primeiro nome (dica: At e Left).
3. Dada uma data, calcule quantos dias faltam para o fim do ano.
4. Monte uma linha de relatório com código (6 posições), descrição (30 posições) e valor formatado, alinhando as colunas com PadR e Transform.

CAPÍTULO 9

Tratamento de erros

> **Neste capítulo:** você aprende a prevenir erros com validações, a capturá-los quando necessário e a fazer funções que informam sucesso ou falha.

9.1 O conceito

Todo programa real encontra problemas: um parâmetro vazio, um registro que não existe, uma divisão por zero. Um programa profissional **antecipa** o que pode dar errado e decide o que fazer. Existem duas defesas:

1. **Prevenir**: validar antes de agir (a defesa preferida).
2. **Capturar**: deixar o erro acontecer, mas interceptá-lo e reagir (o último recurso).

9.2 Prevenir

```
Static Function Divide(nA, nB)
    If nB == 0
        ConOut("Divide: divisão por zero evitada")
        Return 0
    EndIf
Return nA / nB
```

Valide especialmente: parâmetros recebidos, existência de registros (o Found() após uma busca), arrays vazios antes de acessar a posição 1 e valores Nil.

9.3 Capturar com Begin Sequence

```
User Function TesteErro()
    Local bErro := ErrorBlock({|oErr| Break(oErr)})
    Local nRes  := 0
    Local nA    := 10
    Local nB    := 0

    Begin Sequence
        nRes := nA / nB
        ConOut("Não deve chegar aqui")
    Recover Using oErro
        ConOut("Erro capturado: " + oErro:Description)
    End Sequence

    ErrorBlock(bErro)   // restaura o tratador original
Return Nil
```

Como funciona: dentro do Begin Sequence, se ocorrer um erro, o ErrorBlock dispara o Break, que **desvia** a execução para o bloco Recover. Sem erro, o Recover é ignorado. Sempre restaure o ErrorBlock original ao final.

> **Dica:** em versões mais recentes do ambiente também existe a estrutura Try ... Catch ... EndTry, que faz o mesmo de forma mais legível. Confira a disponibilidade e a sintaxe exata na documentação oficial da sua versão antes de adotá-la.

9.4 O que fazer ao capturar um erro

- **Registrar**: grave a mensagem no log (ConOut ou o log do sistema) com informação suficiente para entender depois.
- **Não esconder**: um Recover vazio que engole o erro é perigoso, pois o problema continua e ninguém sabe.
- **Devolver um resultado claro**: retorne .F., um código de erro ou uma mensagem para que quem chamou saiba que falhou.

9.5 Padrão útil: função que informa sucesso ou falha

```
Static Function GravaDado(cCod, cDesc)
    Local lOk := .T.

    If Empty(cCod)
        ConOut("GravaDado: código vazio")
        lOk := .F.
    ElseIf Empty(cDesc)
        ConOut("GravaDado: descrição vazia")
        lOk := .F.
    Else
        // ... aqui entraria a gravação
    EndIf
Return lOk
```

Quem chama decide: If !GravaDado(cCod, cDesc) ... mostra aviso, aborta, tenta de novo.

Exercícios do Capítulo 9

1. Escreva uma função de divisão que nunca gera erro de execução.
2. Provoque propositalmente um erro (acessar aLista[10] em um array de 3 posições) e capture-o com Begin Sequence, mostrando a descrição do erro.
3. Crie uma função que receba um array e devolva o primeiro elemento, tratando o caso de array vazio.

CAPÍTULO 10

Acessando dados no Protheus

> **Neste capítulo:** você lê tabelas, busca registros, grava com segurança e faz consultas com SQL embutido, reunindo toda a lógica dos capítulos anteriores.

10.1 O conceito

Tudo o que você aprendeu até aqui é a **lógica**. Agora vem o motivo de a maioria dos programas existir: **ler e gravar dados** das tabelas do Protheus. Imagine cada tabela como uma planilha enorme: cada **linha** é um **registro** e cada **coluna** é um **campo**. Um programa de dados costuma seguir sempre o mesmo roteiro:

1. Escolher a tabela.
2. Escolher a ordem de leitura (índice).
3. Posicionar no registro desejado (ou no primeiro).
4. Enquanto houver registros: ler os campos, decidir, acumular ou gravar, e ir para o próximo.

Ou seja: **decisão + repetição + variáveis**, exatamente o que você já domina.

10.2 Nomes e campos

No Protheus, as tabelas têm nomes de três caracteres (SA1 = clientes, SB1 = produtos, SC5 = cabeçalho de pedidos de venda, e assim por diante) e os campos levam o prefixo da tabela (A1_COD, A1_NOME, B1_DESC). Para ler um campo, use Alias->CAMPO:

```
ConOut(SA1->A1_NOME)
```

O dicionário de dados do seu ambiente mostra tipo, tamanho e índices de cada campo. Consulte-o sempre; ele é a verdade sobre o seu ambiente.

10.3 Percorrendo uma tabela

```
#Include "TOTVS.CH"

User Function ListaCli()
    Local aArea := GetArea()

    DbSelectArea("SA1")
    SA1->(DbSetOrder(1))       // índice 1
    SA1->(DbGoTop())           // vai para o primeiro registro

    While !SA1->(Eof())        // enquanto não chegar ao fim
        ConOut(AllTrim(SA1->A1_COD) + " - " + AllTrim(SA1->A1_NOME))
        SA1->(DbSkip())        // próximo registro
    EndDo

    RestArea(aArea)
Return Nil
```

Leitura comentada:

- **GetArea() / RestArea(): guardam e restauram** a posição atual das tabelas. É uma regra de convivência: seu programa não pode deixar o ambiente bagunçado para quem o chamou (por exemplo, um ponto de entrada executado no meio de outra rotina).
- **DbSelectArea("SA1"):** seleciona a tabela.
- **SA1->(DbSetOrder(1)):** a sintaxe Alias->(Função()) executa a função **na área do alias indicado**, sem precisar trocar de área.
- **Eof():** verdadeiro quando se passou do último registro.
- **DbSkip():** avança um registro.

> **Atenção:** esquecer o DbSkip é o laço infinito clássico do Protheus.

10.4 Buscando um registro específico: DbSeek

```
Local cCod  := "000001"
Local cLoja := "01"

DbSelectArea("SA1")
SA1->(DbSetOrder(1))

If SA1->(DbSeek(xFilial("SA1") + cCod + cLoja))
    ConOut("Encontrado: " + AllTrim(SA1->A1_NOME))
Else
    ConOut("Cliente não encontrado")
EndIf
```

Pontos de atenção:

- **DbSeek** procura pela **chave do índice selecionado**. A chave que você monta precisa corresponder à ordem e ao tamanho dos campos do índice (veja o índice no dicionário).
- **xFilial("SA1")** devolve a filial correta para a tabela. Em ambientes com várias filiais, **esquecer o xFilial é uma fonte clássica de erros** (buscas que funcionam na sua filial e falham em outra).
- **DbSeek** devolve **.T.** se achou e **.F.** se não achou.

10.5 Gravando: RecLock e MsUnlock

Para alterar ou incluir, o Protheus exige **travar** o registro, gravar e **destravar**. Isso impede que dois usuários gravem ao mesmo tempo no mesmo registro:

```
// Alteração de registro já posicionado
RecLock("SA1", .F.)          // .F. = alterar o registro atual
    SA1->A1_NOME := "NOVO NOME"
SA1->(MsUnlock())            // libera o registro

// Inclusão de registro novo
RecLock("SA1", .T.)          // .T. = incluir
    SA1->A1_FILIAL := xFilial("SA1")
    SA1->A1_COD    := "999999"
    SA1->A1_LOJA   := "01"
    SA1->A1_NOME   := "CLIENTE TESTE"
SA1->(MsUnlock())
```

> **Atenção:** gravar direto nas tabelas padrão com RecLock ignora as validações, os gatilhos e as regras que o sistema aplicaria. Para tabelas do padrão, o caminho recomendado costuma ser o uso de **rotinas automáticas (ExecAuto)** ou das **classes MVC** da própria rotina, que respeitam as regras de negócio. Reserve o RecLock para tabelas customizadas suas ou para casos em que você saiba exatamente o que está fazendo. E sempre teste em ambiente de homologação, nunca direto em produção.

10.6 Consultas com SQL embutido

Quando a tabela é grande, percorrer registro a registro é lento. Nesse caso, delegue o trabalho de filtrar e agrupar ao banco com uma **consulta SQL**. O ADVPL oferece o BeginSql:

```
Local cAlias := GetNextAlias()

BeginSql Alias cAlias
    SELECT A1_COD, A1_NOME
    FROM %table:SA1% SA1
    WHERE SA1.A1_FILIAL = %xFilial:SA1%
      AND SA1.A1_EST = 'MG'
      AND SA1.%NotDel%
    ORDER BY A1_COD
EndSql

While !(cAlias)->(Eof())
    ConOut(AllTrim((cAlias)->A1_COD) + " - " + AllTrim((cAlias)->A1_NOME))
    (cAlias)->(DbSkip())
EndDo

(cAlias)->(DbCloseArea())      // sempre feche a consulta
```

O que cada pedaço faz:

- **GetNextAlias():** gera um nome único para o resultado temporário.
- **%table:SA1%:** o nome físico da tabela no banco.
- **%xFilial:SA1%:** a filial correta para a tabela (equivalente ao xFilial do ADVPL).
- **%NotDel%:** ignora os registros marcados como excluídos, pois o Protheus faz exclusão lógica.
- **(cAlias)->(...):** os parênteses permitem usar o alias guardado em uma variável.
- **DbCloseArea():** libera o resultado. Esquecer isso consome recursos do servidor.

10.7 Qual usar: percorrer ou consultar?

Como regra de bolso: para **poucos registros** e para registros que você já está processando, a navegação por índice (DbSeek/DbSkip) é simples e suficiente. Para **muitos registros**, filtros combinados, somas e agrupamentos, a consulta SQL costuma ser muito mais rápida. E uma regra de ouro de desempenho vale para os dois: **evite consultar o banco dentro de um laço** quando for possível trazer tudo de uma vez antes do laço.

Exercícios do Capítulo 10

1. Liste os 20 primeiros clientes (código e nome). Qual estrutura de repetição você usa, e como garante que são só 20?
2. Dado um código de cliente informado em uma variável, mostre o nome, ou "não encontrado".
3. Conte quantos clientes existem em cada estado. Resolva de duas formas: percorrendo a tabela e com uma consulta com GROUP BY. Compare.
4. (Avançado) Identifique em qual caso é seguro usar RecLock diretamente e em qual é melhor usar rotina automática.

CAPÍTULO 11

Projeto prático: juntando tudo

> **Neste capítulo:** você resolve um problema real do início ao fim: primeiro o algoritmo em português, depois o código, depois a refatoração em funções.

11.1 O problema

Vamos resolver um problema real: **gerar um relatório simples com a quantidade de clientes por estado, em ordem alfabética**. Antes de escrever código, pense em português. Esse é o hábito mais importante deste livro: **primeiro o algoritmo, depois o código**.

11.2 Pensando no algoritmo

1. Preciso de uma lista de pares (estado, quantidade). Isso é um array de arrays.
2. Percorro todos os clientes (laço While até o fim da tabela).
3. Para cada cliente, olho o estado dele. Se o estado ainda **não está** na lista, incluo com quantidade 1. Se **já está**, somo 1 à quantidade.
4. Ordeno a lista por estado.
5. Percorro a lista e mostro cada linha (laço For).

Repare: variáveis, array, While, decisão (If), busca (aScan), ordenação, For e formatação. É tudo o que você aprendeu, em um só programa.

11.3 O código

```
#Include "TOTVS.CH"

User Function RelCliUF()
    Local aArea  := GetArea()
    Local aTotal := {}          // {{cUF, nQtd}, {cUF, nQtd}, ...}
    Local cUF    := ""
    Local nPos   := 0
    Local nI     := 0

    // Passo 2: percorrer os clientes
    DbSelectArea("SA1")
    SA1->(DbSetOrder(1))
    SA1->(DbGoTop())

    While !SA1->(Eof())
        cUF  := SA1->A1_EST
        nPos := aScan(aTotal, {|x| x[1] == cUF})

        // Passo 3: decidir entre incluir o estado ou somar
        If nPos == 0
            aAdd(aTotal, {cUF, 1})
        Else
            aTotal[nPos][2] += 1
        EndIf

        SA1->(DbSkip())
    EndDo

    // Passo 4: ordenar por estado
    aSort(aTotal,,, {|x, y| x[1] < y[1]})

    // Passo 5: mostrar o resultado
    ConOut(Replicate("-", 20))
    For nI := 1 To Len(aTotal)
        ConOut(PadR(aTotal[nI][1], 6) + Transform(aTotal[nI][2], "@E 999,999"))
    Next nI
    ConOut(Replicate("-", 20))

    RestArea(aArea)
Return Nil
```

Observe ainda que este exemplo percorre **todas as filiais** (não filtra A1_FILIAL). Em um caso real, você decidiria se isso é o desejado e, se não for, filtraria pela filial corrente.

11.4 Refatorando em funções

O programa funciona, mas está fazendo tudo em um lugar só. Aplicando o Capítulo 7, o roteiro principal fica assim:

```
User Function RelCliUF()
    Local aArea  := GetArea()
    Local aTotal := ContaPorUF()

    OrdenaPorUF(aTotal)
    ImprimeTotais(aTotal)

    RestArea(aArea)
Return Nil
```

Cada Static Function (ContaPorUF, OrdenaPorUF, ImprimeTotais) cuida de um passo. Agora o roteiro **se lê como a explicação do problema**, e cada parte pode ser testada e corrigida separadamente. Faça essa refatoração como exercício.

11.5 Evoluindo o projeto

Depois que o básico funcionar, evolua aos poucos, um passo por vez, testando entre eles:

1. Mostre também o percentual de cada estado sobre o total (use um acumulador e Round).
2. Ignore clientes com estado vazio (Empty).
3. Troque a leitura registro a registro por uma consulta com GROUP BY (Seção 10.6) e compare os resultados.
4. Faça o relatório aceitar um parâmetro (por exemplo, só uma lista de estados), validando o parâmetro como na Seção 7.4.
5. Proteja o programa contra erros inesperados com o que você viu no Capítulo 9.

CAPÍTULO 12

Boas práticas e depuração

> **Neste capítulo:** você adota hábitos que deixam o código legível e seguro, conhece as armadilhas clássicas e aprende um método para encontrar e corrigir erros.

12.1 Escreva para o próximo leitor

O próximo leitor do seu código, em geral, é **você mesmo daqui a seis meses**. Algumas práticas simples fazem enorme diferença:

- **Nomes claros**: nTotalPedido em vez de nT. Mantenha o prefixo de tipo.
- **Indentação consistente**: cada bloco (If, For, While) recuado em relação ao anterior.
- **Funções pequenas**, com uma responsabilidade.
- **Comentários que explicam o porquê** e um cabeçalho curto no topo do fonte dizendo o que ele faz, quem criou e quando.
- **Sem números e textos "mágicos"**: se 0.15 é uma alíquota, dê um nome a ela ou use um parâmetro do sistema (SuperGetMV) em vez de deixar escrito no meio da conta.
- **Declare variáveis no início** e inicialize todas.
- **Escopo mínimo**: Local por padrão.

12.2 Armadilhas clássicas (lista de conferência)

1. Usar = onde queria ==.
2. Confundir := (atribuir) com == (comparar).
3. Comparar texto de campo sem AllTrim.
4. Esquecer o DbSkip() em um While de tabela (laço infinito).
5. Esquecer o xFilial na busca ou na consulta.

6. Esquecer RestArea e deixar o ambiente alterado.
7. Acessar posição de array inexistente (aLista[0] ou além do Len).
8. Não tratar Nil em parâmetros opcionais.
9. Esquecer MsUnlock depois de RecLock.
10. Não fechar áreas de trabalho de consultas (DbCloseArea).
11. Usar Public onde Local bastava.
12. Consultar o banco dentro de laços sem necessidade.

12.3 Depurando: como achar um erro

Erros fazem parte do ofício. O que separa o iniciante do experiente é o método:

1. **Reproduza** o problema de forma confiável. Sem reproduzir, não há como saber que você consertou.
2. **Leia a mensagem de erro** com calma: ela costuma dizer a função, a linha e o motivo (por exemplo, *array out of bounds* ou *type mismatch*).
3. **Isole**: reduza o problema até o menor trecho que ainda falha.
4. **Observe os valores**: use ConOut para ver o que as variáveis realmente contêm em cada etapa. Muitas vezes o valor não é o que você imaginava (um espaço sobrando, um tipo diferente).
5. **Use o depurador** do seu ambiente (TDS ou VS Code): coloque pontos de parada (*breakpoints*), execute passo a passo e acompanhe as variáveis. É a ferramenta mais poderosa de aprendizado, e vale a pena aprender a usá-la cedo.
6. **Mude uma coisa por vez** e teste. Várias mudanças ao mesmo tempo escondem a causa.
7. **Confirme a correção** e pense: este mesmo erro pode existir em outro lugar?

12.4 Testando sem medo

Teste sempre os **casos de borda**: array vazio, texto em branco, valor zero, valor negativo, o primeiro e o último registro, Nil. É aí que os erros moram. Faça os testes em ambiente de desenvolvimento ou homologação, nunca em produção.

12.5 Desempenho em uma frase

Primeiro faça funcionar e ficar legível; depois, se estiver lento, **meça** onde está o gargalo antes de otimizar. Quase sempre o problema é o acesso a dados (muitas consultas, laço dentro de laço, falta de filtro), e não a lógica.

Palavras finais

Não existe atalho para aprender a programar além de **escrever muito código, errar, depurar e tentar de novo**. Se um capítulo parecer difícil, volte ao conceito (a ideia em português) antes de olhar a sintaxe: a linguagem muda, mas sequência, decisão, repetição, variáveis e funções são os mesmos blocos em qualquer lugar. Quando esses blocos estiverem claros na sua cabeça, o ADVPL passa a ser só uma questão de saber o nome das ferramentas, e isso você aprende fazendo.

Bons estudos!

APÊNDICE A

Roteiro de estudos

Um caminho sugerido, em ritmo de poucas horas por semana:

Semana	Foco	Entrega
1	Capítulos 1 e 2	5 programas curtos com variáveis e conversões
2	Capítulos 3 e 4	Exercícios de decisão (desconto, situação do aluno)
3	Capítulo 5	Somatórios, fatorial, tabuada, FizzBuzz
4	Capítulo 6	Programas com arrays e matrizes
5	Capítulo 7	Refatorar os programas anteriores em funções
6	Capítulos 8 e 9	Manipulação de texto e data, e validações
7	Capítulo 10	Leitura de tabelas e uma consulta SQL
8	Capítulos 11 e 12	Projeto do relatório e evoluções

> **Dica:** mantenha um **caderno de padrões** com os trechos que você mais reutiliza (percorrer tabela, buscar registro, montar consulta). Ele vira o seu manual pessoal.

APÊNDICE B

Glossário

- **Algoritmo:** sequência de passos que resolve um problema.
- **Alias:** apelido da área de trabalho onde uma tabela está aberta (por exemplo, SA1).
- **Array:** variável que guarda vários valores em posições numeradas.
- **Bloco de código:** trecho de código guardado em uma variável, no formato {|x| ...}.
- **Escopo:** o alcance de visibilidade de uma variável (Local, Static, Private, Public).
- **Filial:** divisão da empresa no Protheus; xFilial devolve a filial correta de cada tabela.
- **Função:** bloco de código com nome, que recebe parâmetros e devolve um valor.
- **Índice:** ordem de leitura de uma tabela, usada em buscas como DbSeek.
- **Laço:** estrutura que repete um bloco (For, While).
- **Parâmetro:** dado que uma função recebe ao ser chamada.
- **Ponto de entrada:** função com nome pré-definido que o sistema padrão executa em momentos específicos.
- **Referência:** forma de passar uma variável em que a função altera o original (@).
- **Registro:** uma linha de uma tabela.
- **RPO:** repositório de objetos onde ficam os programas compilados.
- **Variável:** espaço nomeado na memória para guardar um valor.

APÊNDICE C

Gabarito comentado

Resoluções de exercícios selecionados. Tente resolver sozinho antes de consultar.

Capítulo 3, exercício 1 (desconto e imposto)

```
Local nPreco   := 200
Local nComDesc := nPreco * (1 - 0.10)
Local nFinal   := nComDesc * (1 + 0.05)   // 189
```

O imposto incide sobre o valor **já com desconto**, por isso a ordem dos cálculos importa.

Capítulo 4, exercício 1 (desconto progressivo)

Teste do **maior** limite para o **menor**:

```
If nCompra >= 1000
    nDesc := 0.15
ElseIf nCompra >= 500
    nDesc := 0.10
ElseIf nCompra >= 100
    nDesc := 0.05
Else
    nDesc := 0
EndIf
```

Se você testasse >= 100 primeiro, todo mundo acima de 100 cairia no primeiro bloco e nunca chegaria aos descontos maiores.

Capítulo 4, exercício 3 (ano bissexto)

```
Local lBissexto := (nAno % 400 == 0) .Or. (nAno % 4 == 0 .And. nAno % 100 <> 0)
```

Capítulo 5, exercício 2 (fatorial)

```
Local nFat := 1
Local nI   := 0

For nI := 2 To nN
    nFat *= nI
Next nI
```

O acumulador começa em **1** (e não em 0), pois é uma multiplicação: começar em 0 zeraria tudo.

Capítulo 5, exercício 3 (rendimento)

O laço adequado é o While, porque você não sabe quantas voltas serão necessárias; só sabe a condição de parada.

```
Local nSaldo := 1000
Local nMeses := 0

While nSaldo <= 1500
    nSaldo *= 1.01
    nMeses++
EndDo
```

Capítulo 5, exercício 4 (FizzBuzz)

Teste primeiro o caso mais específico (múltiplo de 3 **e** de 5), senão ele seria capturado pelos casos mais gerais.

```
For nI := 1 To 20
    Do Case
        Case nI % 15 == 0
            ConOut("FizzBuzz")
        Case nI % 3 == 0
            ConOut("Fizz")
        Case nI % 5 == 0
            ConOut("Buzz")
        Otherwise
            ConOut(cValToChar(nI))
    EndCase
Next nI
```

Capítulo 6, exercício 1 (soma e média)

```
Local aNum  := {4, 8, 15, 16, 23}
Local nSoma := 0
Local nI    := 0

For nI := 1 To Len(aNum)
    nSoma += aNum[nI]
Next nI

ConOut("Média: " + cValToChar(nSoma / Len(aNum)))
```

Capítulo 7, exercício 1 (média em função)

```
Static Function Media(aNumeros)
    Local nSoma := 0
    Local nI    := 0

    If Len(aNumeros) == 0
        Return 0
    EndIf

    For nI := 1 To Len(aNumeros)
        nSoma += aNumeros[nI]
    Next nI
Return nSoma / Len(aNumeros)
```

Capítulo 8, exercício 1 (limpar CPF)

```
Local cCPF := "123.456.789-09"
cCPF := StrTran(cCPF, ".", "")
cCPF := StrTran(cCPF, "-", "")     // "12345678909"
```

Referências e leituras recomendadas

- **TDN — TOTVS Developer Network** (tdn.totvs.com): documentação oficial de funções, comandos, classes e pontos de entrada do Protheus. É a fonte a consultar sempre que houver dúvida sobre a sintaxe ou o comportamento na sua versão.
- **Documentação do seu ambiente**: o dicionário de dados (tabelas, campos e índices) e a versão do Application Server e da lib em uso determinam detalhes que variam de cliente para cliente.
- **Comunidade**: fóruns e grupos de desenvolvedores Protheus são um ótimo lugar para trocar experiências. Ao pedir ajuda, descreva o problema, mostre o trecho de código e a mensagem de erro completa.

Sobre o autor

Domingos Elias de Oliveira Neto é profissional de tecnologia da informação e trabalha com o ERP TOTVS Protheus. Este livro nasceu do seu estudo de ADVPL e do desejo de compartilhar esse caminho com quem está começando.